

CUSTOMER FEATURE GUIDE

Aether

Universal runtime portability.

Deploy once, then move workloads across Podman, Kubernetes, KubeVirt, and Metal3 from a single YAML spec — no infrastructure rewrites.

by ZyvorAI Labs

What is Aether?

Aether is a universal runtime control plane: one workload spec describes what to run, and Aether deploys it to the right runtime, explains why, and migrates it between runtimes with production strategies. It ships a Rust CLI, an interactive TUI, and a React web dashboard over a shared control plane — think of it as Terraform for where your workloads run, not just what they are.

4

Runtimes (Podman, K8s, KubeVirt, Metal3)

16

Migration paths

5

Migration strategies

65+

CLI commands

What's inside this guide

Getting started — access & first workflows	start here
1 · One Spec, Four Runtimes	7 features
2 · Production Migration Engine	7 features
3 · Intent Decision Engine	7 features
4 · Discovery, Assessment & Cloud Exit	7 features
5 · Operations & Lifecycle	7 features
6 · Observability & Cost Insight	7 features
7 · AI Ops & Autonomy	7 features
8 · Security, Policy & Compliance	7 features
9 · Confidential Computing (Ragnarok)	6 features
10 · Integrations & Extensibility	8 features
Support & contact	→

Access & first workflows

How to reach Aether and get your first result. Assumes it has been deployed for you.

How to access it

WEB

Run `aether serve --port 8080` and open `http://localhost:8080` for the React dashboard (default port is 5090 if `--port` is omitted). It offers a ⌘K / Ctrl+K command palette, live SSE real-time updates (no polling), a runtime-fabric topology graph, tabbed workload detail, and the Ask Aether copilot rail.

CLI

The `aether` binary is the primary interface (65+ commands). Core loop: `aether init` (first-run wizard), `aether validate --spec workload.yaml`, `aether build`, `aether run --runtime kube`, `aether list`, `aether status <name>`, `aether logs <name> --follow`, `aether migrate <name> <target> --strategy blue-green`, and `aether policy-check --policy production`. Global flags include `-s/--spec`, `-n/--namespace`, `-o/--output {table,json,yaml,wide}`, and `-y/--yes` for automation.

API

The same `aether serve` process exposes a REST API under `/api` on the serve host (e.g. `GET /api/workloads`, `POST /api/workloads`, `GET /api/workloads/{name}/logs`, `POST /api/cost`, `GET /api/command-center/briefing`, `GET/POST /api/rbac/keys`). Responses are `{success, data, error}` JSON; `GET /health` and `GET /metrics` are public.

LOGIN

Dashboard default credentials are `admin` / `aether`. API auth is off by default; set `AETHER_API_KEY` to require an `Authorization: Bearer <key>` header on `/api/*`. Enterprise SSO is available via `AETHER_OIDC_*` / `AETHER_SAML_*`.

NEEDS

Linux (x86_64/aarch64) with Rust 1.75+ and Podman 4.0+; build with `cargo build --release`, install the `aether` binary, then run `aether init`.

Your first workflows



Deploy your first workload

1. Scaffold a spec: `aether template web-app --workload-name hello-web --output workload.yaml`.
2. Validate it: `aether validate` (or `aether validate --spec workload.yaml`).
3. Build the image: `aether build`.
4. Deploy auto-selecting the runtime: `aether run` (or pin one with `aether run --runtime podman`).
5. Confirm and watch: `aether status hello-web` then `aether logs hello-web --follow`.



Migrate a live workload across runtimes

1. Assess the move first: `aether migration-advice hello-web kubernetes`.
2. Cut over with zero downtime: `aether migrate hello-web kube --strategy blue-green`.
3. For critical apps use gradual traffic shift: `aether migrate hello-web kube -s rolling`.
4. Verify the new runtime: `aether status hello-web` then `aether diff hello-web`.
5. On trouble, undo: `aether rollback hello-web` (or migrate with `--no-rollback` disabled by default).



Explain and choose a runtime with intent scoring

1. Rank every eligible runtime with reasons: `aether recommend` .
2. Evaluate the spec's intent goals and weights: `aether intent` .
3. Compare cost, capabilities, and limits side by side: `aether compare` .
4. Deploy to the chosen runtime: `aether run --runtime <winner>` .
5. Visualize the trade-offs in the dashboard: Workload detail → Scoring (Intent Debugger radar).



Deploy a multi-workload stack with compose

1. Author `aether-compose.yaml` with workloads: entries, per-workload `runtime` , `env` , and `depends_on` ordering.
2. Validate the stack: `aether compose validate` .
3. Deploy all workloads in dependency order: `aether compose up` (override runtime with `--runtime kube`).
4. Preview without applying: `aether compose up --dry-run` .
5. Tear the stack down in reverse order: `aether compose down` .



Discover a cluster and plan a cloud exit

1. Save and test a cluster connection: `aether connection add` .
2. Inventory what is running: `aether discover start` then `aether inventory applications` .
3. Assess an app's portability: `aether assess payments` and inspect deps with `aether deps show` .
4. Generate a versioned plan: `aether plan create` (emits a MigrationPlan CRD).
5. Rehearse safely with a shadow deploy: `aether move plan` → `aether move start` → `aether move cutover` (or `aether move rollback`).



Operate and observe the fleet

1. Launch the terminal dashboard: `aether tui` , or the web UI: `aether serve --port 8080` → `http://localhost:8080`.
2. Watch health continuously: `aether orchestrate watch --interval 10` .
3. Detect and fix config drift: `aether drift hello-web --reconcile` .
4. Estimate spend across clouds: `aether cost --provider all` .
5. Snapshot state before risky changes: `aether backup -n before-upgrade` .

One Spec, Four Runtimes

A single Aether workload spec validates and deploys everywhere — from a laptop container to bare metal.

Runtime	Best for
Podman	Local dev and edge containers
Kubernetes	Cluster orchestration, services, scaling
KubeVirt	VM isolation and GPU passthrough
Metal3	Bare-metal performance



Universal Workload Spec

Describe a workload once in the aether/v1 YAML schema — image, resources, ports, persistence, secrets, and intent.

Stop maintaining four different manifest dialects for the same app.

How · CLI: scaffold with `aether template web-app -o workload.yaml`, then edit the `aether/v1` YAML; Web UI: Templates page.



Validate Before Deploy

Check spec syntax, intent, and per-runtime compatibility before anything ships.

Catch misconfiguration at author time, not in production.

How · CLI: `aether validate` (or `aether validate -s my-app.yaml`).



Run Anywhere

Deploy a spec to Podman, Kubernetes, KubeVirt, or Metal3 — with the runtime auto-selected or pinned by flag.

The same command targets local dev, clusters, VMs, and bare metal.

How · CLI: `aether run` (auto-select) or `aether run --runtime kube`; Web UI: Workloads page; REST: `POST /api/workloads`.



Podman & Local Dev

Run containers locally or at the edge through the Podman adapter with the identical spec you ship to prod.

True dev/prod parity from the first line of YAML.

How · CLI: `aether run --runtime podman` (alias `container`).



Kubernetes Native

Generate Deployments, Services, Ingress, HPA, ConfigMaps, and Secrets from one spec via the Kubernetes adapter.

Full cluster orchestration without hand-writing manifests.

How · CLI: `aether run --runtime kube` (aliases `kubernetes` / `k8s`); set namespace with `-n` or `AETHER_NAMESPACE`.



KubeVirt VMs

Deploy the workload as a KubeVirt virtual machine for strong isolation and GPU passthrough.

Move a container to a VM when you need harder boundaries — no rewrite.

How · CLI: `aether run --runtime kubvirt` (alias `vm`).



Metal3 Bare Metal

Target bare-metal hosts through the Metal3 adapter for maximum performance.

Reach dedicated hardware from the same control plane as everything else.

How · CLI: `aether run --runtime metal3` (aliases `metal` / `bare-metal`).

SECTION 2

Production Migration Engine

Move a running workload between any two runtimes with health gates, connection draining, and automatic rollback.



16 Migration Paths

Migrate across every ordered pair of the four runtimes with a single migrate command.

Escape a legacy VM or container stack without a re-platforming project.

How · CLI: `aether migrate <name> <target-runtime>` (e.g. `aether migrate hello-web kube`).



Blue-Green Cutover

Stand up the target (green) while the source (blue) keeps serving, health-gate it, then cut over.

Zero-downtime moves that only switch traffic once the target is proven healthy.

How · CLI: `aether migrate hello-web kube --strategy blue-green` (the default strategy).



Rolling & Immediate

Choose incremental rolling steps, an immediate stop-then-start, or canary — per migration.

Match the migration risk profile to each workload.

How · CLI: `aether migrate hello-web kube -s rolling` or `-s immediate`.



Automatic Rollback

On a failed health gate the engine cleans up the target and restores the source automatically.

A failed migration leaves you where you started, not half-migrated.

How · CLI: on by default; disable with `aether migrate ... --no-rollback`, or undo manually with `aether rollback <name>`.



Health Gates & Draining

Exponential-backoff health validation and connection draining (up to a 30s cap) guard every cutover.

In-flight requests finish before the old instance goes away.

How · CLI: runs inside `aether migrate`; skip the validation delay with `--no-validation`.



Migration Trace

Stream every phase — snapshot, build-target, health-gate, drain, update-state — to stderr for debugging.

See exactly where a migration is and why, step by step.

How · CLI: streamed to stderr during `aether migrate`; add `-v` for verbose phase output.



AI Migration Advice

Get an AI assessment of moving a named workload to a target runtime before you commit.

Understand the risks and gains of a move up front.

How · CLI: `aether migration-advice hello-web kube`; Web UI: Migrations.

Migration is designed for stateless and container-image portability. Persistent volume data is not automatically carried across runtimes — see caveats.

Intent Decision Engine

Score and rank runtimes on cost, performance, reliability, and availability — with reasons, not a black-box pick.



Explainable Placement

aether decide ranks every eligible runtime with per-dimension scores and human-readable reasons.

Stop guessing which runtime fits — see the math behind the choice.

How · CLI: `aether recommend` (or `aether decide`);
Web UI: Workload detail → Scoring.



Four-Dimension Scoring

Each runtime is scored 0–1 on cost, performance, reliability, and availability with configurable weights.

Tune the ranking to your organization's priorities.

How · CLI: `aether recommend` ; Web UI: AI Engine page.



Intent Multipliers

An intent block (goal, SLA, budget, resilience, compliance) reshapes the weights toward your objective.

Encode 'low-latency' or 'cost-optimized' once and let the engine honor it.

How · CLI: `aether intent` (evaluates the spec's `intent` block goals and scoring).



Workload Classification

Specs are auto-classified (stateless, stateful, GPU, bare-metal, batch) to filter ineligible runtimes.

GPU jobs never get scored onto runtimes that can't run them.

How · CLI: applied automatically during `aether recommend` ; inspect resource fit with `aether profile` .



Runtime Compare

Side-by-side comparison of cost, capabilities, and limitations across all four runtimes for a spec.

One view to justify a placement decision to your team.

How · CLI: `aether compare` ; Web UI: AI Engine page.



Intent Debugger (Radar)

The dashboard renders scores as a radar chart so you can see trade-offs at a glance.

Turn placement scoring into a picture stakeholders understand.

How · Web UI: Workload detail → Scoring tab (pure-SVG radar chart across the four axes).



Affinity Learning

Recommend runtimes per workload class from a learned compatibility matrix and usage statistics.

Recommendations sharpen as the platform learns your fleet.

How · CLI: `aether affinity recommend` / `aether affinity matrix` / `aether affinity stats` ; Web UI: Affinity page.

Discovery, Assessment & Cloud Exit

Connect to existing clusters, inventory their apps, score portability, and generate a cloud-exit plan.



Cluster Connections

Save and test connections to EKS, AKS, GKE, OpenShift, Rancher, Tanzu, k3s, RKE2, Podman, or Compose.

Point Aether at what you already run — no agents to install first.

How · CLI: `aether connection add` (save and test a cluster connection); Web UI: Fleet / Clusters.



Application Discovery

Scan a connected cluster and persist an inventory snapshot of its applications and workloads.

Get a real map of what's running before you plan a move.

How · CLI: `aether discover start` then `aether inventory applications`.



Dependency Graph

Surface the dependency graph for a discovered application across its services and data stores.

See what breaks together before you migrate anything.

How · CLI: `aether dependency graph / aether deps show` (and `aether deps impact`); Web UI: Dependencies page.



Portability Assessment

Assess an application's migration readiness and complexity, with an optional target-cluster preflight.

Know which apps are easy wins and which need work.

How · CLI: `aether assess <app>` (e.g. `aether assess payments`).



Migration Plan CRD

Generate a MigrationPlan (`aether.zyvor.dev/v1alpha1`) with a recommended or overridden strategy.

Turn an assessment into a reviewable, versioned plan artifact.

How · CLI: `aether plan create` (emits the MigrationPlan `aether.zyvor.dev/v1alpha1` CRD).



Cloud Exit Report

Produce a Cloud Exit Assessment report for a connection in Markdown, JSON, or text.

A shareable, evidence-backed case for leaving a cloud.

How · CLI: generate the Cloud Exit Assessment from an assessed connection with `-o markdown|json|text`; Web UI: Migrations.



Move (Shadow Deploy)

Mirror images and shadow-deploy an app to a target with no external traffic, then cut over or roll back.

[Rehearse a cross-cluster move safely before flipping DNS.](#)

How · CLI: `aether move plan` → `aether move start` → `aether move cutover` (or `aether move rollback`); track with `aether move status`.

Operations & Lifecycle

Day-2 workload operations — inspect, scale, restart, back up, roll back, and drain nodes from one tool.



Logs, Exec & Port-Forward

Tail logs with follow mode, exec into containers, forward ports, and copy files across runtimes.

[The kubectl toolbox for every runtime, in one CLI.](#)

How · CLI: `aether logs <name> --follow`, `aether exec <name> "ls -la"`, `aether port-forward <name> 8080:80`.



Scale & Restart

Scale a Kubernetes workload to a target replica count or trigger a rolling restart.

[Routine scaling actions without leaving Aether.](#)

How · CLI: `aether scale my-app <replicas>`; Web UI: Workload detail → Restart.



Health-Aware Orchestration

Register workloads for health monitoring, watch at an interval, and trip or reset circuit breakers.

[Continuous health signal that feeds migrations and auto-rollback.](#)

How · CLI: `aether orchestrate watch --interval 10`, `aether orchestrate status`, `aether orchestrate summary`.



Backup & Restore

Snapshot workload state locally or to a remote endpoint and restore or merge it back.

[State you can recover after a bad change or a lost host.](#)

How · CLI: `aether backup -n <name>`, `aether restore ./backup.json [--merge]`; REST: `POST /api/backups`.



Snapshot Rollback

Roll a workload back to any listed snapshot version of its stored state.

[Undo a deployment to a known-good point in seconds.](#)

How · CLI: `aether snapshot list` then `aether snapshot rollback` (or `aether rollback <name>`).



Node Maintenance

Cordon, uncordon, and safely drain Kubernetes nodes for maintenance windows.

[Take hardware offline without evicting workloads by hand.](#)

How · CLI: `cordon` / `uncordon` / `drain` via Aether's node-maintenance commands; Web UI: Fleet.



Watch & Auto-Redeploy

Watch a spec file and auto-redeploy on change, or reconcile a whole directory of specs.

[A fast inner loop that keeps live state matching your files.](#)

How · CLI: `aether watch` (single spec) or `aether deploy ./specs/` (reconcile a directory).

Observability & Cost Insight

A k9s-level view of the fleet — metrics, events, audit, drift, cost, and SLA — across CLI, TUI, and web.



Web Dashboard

A 19-page React dashboard with SSE real-time updates, command palette (⌘K), and a live runtime-fabric topology graph.

One pane of glass for the whole runtime fleet.

How · CLI: `aether serve --port 8080`, then open `http://localhost:8080`.



Interactive TUI

A k9s-style terminal dashboard for real-time monitoring, logs, and workload navigation.

Full situational awareness without leaving the terminal.

How · CLI: `aether tui`.



Prometheus Metrics

Export workload and control-plane metrics in Prometheus format for your existing stack.

Plug Aether straight into Grafana and Alertmanager.

How · CLI: `aether metrics`; REST: `GET /metrics`.



Events & Audit Trail

Filterable event stream by severity plus a tamper-evident audit trail of every action.

Know what changed, when, by whom — with integrity you can prove.

How · CLI: `aether events`, `aether audit`; Web UI: Events / Audit pages.



Drift Detection

Detect configuration drift between spec, stored state, and live runtime — with optional auto-reconcile.

Silent config drift becomes a visible, fixable signal.

How · CLI: `aether drift <name>` (add `--reconcile` to fix); Web UI: Drift page.



Cost Estimation

Estimate workload cost across AWS, Azure, GCP, DigitalOcean, and Linode with spot/reserved discounts.

Compare the price of a workload before you place it.

How · CLI: `aether cost --provider all`; Web UI: Cost page; REST: `POST /api/cost`.



SLA Compliance

Set SLA tiers per workload and check observed uptime, latency, error rate, and restarts against them.

[Turn availability promises into monitored, reportable targets.](#)

How · CLI: `aether sla set` / `aether sla check` / `aether sla report` ; Web UI: SLA page.

SECTION 7

AI Ops & Autonomy

Ask Zeus, the terminal and dashboard AI assistant, plus an intelligence layer that predicts, heals, and advises.



Ask Zeus Copilot

An interactive AI ops assistant in the terminal and a permanent dashboard rail for natural-language questions.

[Ask the platform what's wrong and what to do about it.](#)

How · CLI: `aether ask "why is my-app unhealthy?"` ; Web UI: Copilot rail (xl+) or full-page `/copilot`.



Predictive Scaling

Recommend scaling actions and forecast capacity from observed workload behavior.

[Size ahead of demand instead of reacting to alerts.](#)

How · CLI: `aether scaling-advice` ; Web UI: AI Engine page.



Self-Healing & Remediation

An autonomy layer detects anomalies and proposes or runs remediation for failing workloads.

[Common failures get fixed before they page a human.](#)

How · Web UI: AI Studio / Labs autonomy panels (opt-in previews).



Log Anomaly Analysis

Analyze a workload's logs for anomalies and recurring patterns.

[Find the needle in noisy logs without a query language.](#)

How · CLI: `aether analyze-logs <name>`.



Digital Twin

Model the fleet as a digital twin to reason about capacity, placement, and change impact.

[Test a decision against a model before touching production.](#)

How · Web UI: Labs (digital twin panel).



Command Center Briefing

A narrative fleet briefing summarizing health, intent, SLA, and the next recommended actions.

[Start the day with a plain-language state of the fleet.](#)

How · Web UI: Overview / Command Center; REST: `GET /api/command-center/briefing`.



Autonomous FinOps & SRE

Agent panels for autonomous placement, FinOps budget enforcement, and SRE reliability workflows.

[Delegate routine cost and reliability toil to guided agents.](#)

How · Web UI: AI Studio / Labs agent panels (opt-in previews).

AI features connect to an OpenAI-compatible LLM or the Forge inference gateway; several autonomy panels ship as opt-in Labs previews.

Security, Policy & Compliance

Encrypted secrets, enterprise SSO, RBAC, policy gates, and supply-chain evidence built into the control plane.



AES-256-GCM Secrets

Encrypt all secret values at rest with authenticated AES-256-GCM, with a managed rotation lifecycle.

Secrets stay protected on disk and rotate on schedule.

How · CLI: `aether secrets set <ns> <key> <value>` (set `AETHER_SECRET_KEY` first); rotate with `aether secrets rotate` / audit with `aether secrets audit`.



Enterprise SSO

Authenticate via LDAP, SAML, or OIDC with role mapping from your identity provider.

Plug Aether into the SSO your organization already runs.

How · Config: set `AETHER_OIDC_*` or `AETHER_SAML_*` env vars before `aether serve` (set `AETHER MOCK_IDP=1` for local testing).



RBAC & Audit Export

Role-based access control across API and UI, with exportable audit logs for compliance.

Least-privilege access with an exportable paper trail.

How · REST: `GET/POST /api/rbac/keys` and `/api/rbac/keys/revoke` (Admin role); CLI: `aether audit` exports the trail.



Policy Gate on Deploy

Enforce production/development policy sets — or a custom OPA/Rego policy — before a workload ships.

Non-compliant workloads are blocked at deploy, not audited after.

How · CLI: `aether policy-check --policy production` (runs automatically on deploy; bypass with `--skip-policy`); Web UI: Policy page.



SBOM Export & Verify

Produce and validate CycloneDX software bills of materials for workloads.

Supply-chain evidence auditors and customers can verify.

How · CLI: `aether sbom export` (CycloneDX).



TLS Lifecycle

Serve the API over HTTPS with self-managed certificate regeneration and opt-in auto-renewal.

Encrypted endpoints without manual cert babysitting.

How · CLI: `aether serve --tls-cert <path> --tls-key <path>` for HTTPS.



Hardened Control Plane

API key auth, CORS protection, rate limiting, input validation, atomic state locking, and hardened file permissions.

Production-safe defaults across the whole surface.

How · Config: enable bearer auth with `AETHER_API_KEY` ; rate limiting, input validation, and atomic state locking apply around `aether serve` .

Confidential Computing (Ragnarok)

Run and migrate attested, measured workloads on trusted execution environments with sovereign-cloud controls.



Measured Image Catalog

Sign VM disk images into a verified catalog and verify a file hash or launch digest against it.

Only known, signed images ever launch.

How · CLI: `aether confidential image` (sign into and verify against the measured-image catalog).



Attestation & Trust

Verify TEE attestation verdicts (SEV-SNP, TDX) and gate placement on the trust result.

Prove a workload runs on genuine confidential hardware.

How · CLI: `aether confidential placement` (gate on the SEV-SNP/TDX verdict); Web UI: Workload detail `?tab=trust`.



Tenant Isolation Check

Validate a workload spec against tenant isolation policy before scheduling.

Catch isolation violations before a tenant is exposed.

How · CLI: `aether confidential placement` validates the spec against tenant-isolation policy before scheduling.



GuestKit Offline Inspection

Inspect a VM image offline in pre-launch, offline-policy, post-shutdown, or attested-repair modes.

Assurance on a VM's contents without booting it.

How · CLI: `aether confidential guestkit` (pre-launch / offline-policy / post-shutdown / attested-repair modes).



Confidential Migration

Plan and track encrypted, attested migration of confidential workloads (confidential blue-green).

Move sensitive VMs between runtimes without breaking the trust chain.

How · CLI: confidential blue-green via `aether migrate` on an attested workload; Web UI: Migrations.



Sovereign Compliance Check

Check a workload against sovereign-cloud residency and compliance constraints.

Keep regulated workloads inside the boundaries they require.

How · CLI: `aether confidential placement` checks the spec against sovereign residency and compliance constraints.

Confidential features require compatible TEE hardware (AMD SEV-SNP or Intel TDX) and a Kata/KubeVirt confidential runtime.

Integrations & Extensibility

Slot Aether into your platform — GPU, storage, GitOps, packaging, plugins, edge fleets, and multi-tenant hosting.



Forge GPU/AI

Read live GPU capacity, node inventory, placement recommendations, and cost from the Forge control plane.

Reason about GPU/AI infrastructure in the same tool as everything else.

How · CLI: `aether forge nodes` / `aether forge recommend` / `aether forge cost` / `aether forge stats` ; Web UI: GPU / Forge page.



Atlas Storage

Provision policy-driven persistent volumes (Ceph, NFS, ZFS) through Atlas with snapshot and clone support.

Intent-based storage instead of hand-rolled PVCs.

How · CLI: `aether storage list` / `aether storage snapshot` / `aether storage clone` / `aether storage restore` ; Web UI: Storage page.



GitOps Reconcile

Bind a Git repo and branch, sync manually or auto-reconcile on a poller in serve mode.

Declarative, Git-sourced fleet state with drift correction.

How · Web UI: GitOps page (bind repo/branch, sync or auto-reconcile while `aether serve` runs).



Helm & Package Deploy

Export any workload as a Helm chart, or install via Docker images and deb/rpm packages.

Deliver Aether-managed workloads through your existing pipeline.

How · CLI/Web UI: export a workload as a Helm chart; distribute Aether itself via `ghcr.io/ssahani/aether` Docker image or deb/rpm packages.



Plugin System

Discover and register runtime plugins from manifests to extend Aether beyond the built-in adapters.

Add new runtimes and behaviors without forking the core.

How · CLI: `aether plugin discover`, `aether plugin register ./manifest.json`, `aether plugin list` ; REST: `POST /api/plugins/discover`.



Webhooks & Notifications

Route severity-filtered notifications to webhook channels with a retry queue and test/flush controls.

Wire fleet events into Slack, PagerDuty, or any endpoint.

How · CLI: `aether webhook add`, `aether webhook test`, `aether webhook queue`, `aether webhook flush`.



Edge Fleet & Federation

Register edge sites that drain a reconcile queue from the control plane, with fleet federation across clusters.

[Manage disconnected and edge sites from one control plane.](#)

How · CLI: `aether edge-agent` (drains the reconcile queue at an edge site); Web UI: Fleet.



Hosted Multi-Tenancy

A hosted control-plane foundation with tenant registry, switcher, managed upgrades, and billing/metering.

[Run Aether as a multi-tenant service for many teams.](#)

How · Web UI: hosted control-plane tenant registry and switcher (Settings).

Support & next steps

Support & contact

Aether is developed by **ZyvorAI Labs**. For onboarding, a proof-of-concept, or a guided demo, contact your account team at **info@zyvor.dev**.

Good to know: Aether's portability guarantees are strongest for stateless and container-image workloads: migration rebuilds the app image for the target runtime and updates workload state atomically, but persistent volume data is not automatically carried across runtimes and blue-green connection draining is capped at ~30s. Integrations are optional and mostly read-only — Forge (GPU) and Atlas (storage) are off unless configured, and the hosted multi-tenant control plane ships today as a foundation with fleet federation still maturing. Confidential computing requires compatible TEE hardware (AMD SEV-SNP / Intel TDX) and a Kata/KubeVirt confidential runtime, and several AI autonomy panels are opt-in Labs previews that depend on an external or Forge-hosted LLM.

Aether

Deploy once, then move workloads across Podman, Kubernetes, KubeVirt, and Metal3 from a single YAML spec — no infrastructure rewrites.

info@zyvor.dev · zyvor.dev